06 . Utilize the K-Means clustering algorithm to segment customers based on their purchasing behaviour. Apply the Elbow Method to determine the optimal number of clusters that best represent distinct customer segments for targeted marketing strategies.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import *
```

In [3]:
```
df = pd.read_csv('Wholesale_Customers.csv')
print("Dataset Shape :", df.shape)
df.sample(5)
```

```
Dataset Shape : (440, 9)
```

Out[3]:

	Channel	Region	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen	Cluster
235	1	1	1838	6380	2824	1218	1216	295	1
371	1	3	20105	1887	1939	8164	716	790	1
152	1	3	18840	1371	3135	3001	352	184	1
80	1	3	7005	829	3009	430	610	529	1
362	1	3	2121	1601	2453	560	179	712	1

In [4]:
```
if df.isnull().sum().sum() > 0:
    print("Missing Values Present")
    print(df.isnull().sum())
else :
    print("No Missing Values Found")
```

```
No Missing Values Found
```

In [5]:
```
df.columns
```

Out[5]:
```
Index(['Channel', 'Region', 'Fresh', 'Milk', 'Grocery', 'Frozen',
       'Detergents_Paper', 'Delicassen', 'Cluster'],
      dtype='object')
```

In [6]:
```
x = df.drop(['Channel', 'Region', 'Cluster'], axis=1)
```

In [7]:
```
scaler = StandardScaler()
x_scale = scaler.fit_transform(x)
```

In [8]:
```
wcss = []

for i in range(1, 11) :
    kmeans = KMeans(n_clusters=i, n_init=10, random_state=42)
    kmeans.fit(x_scale)

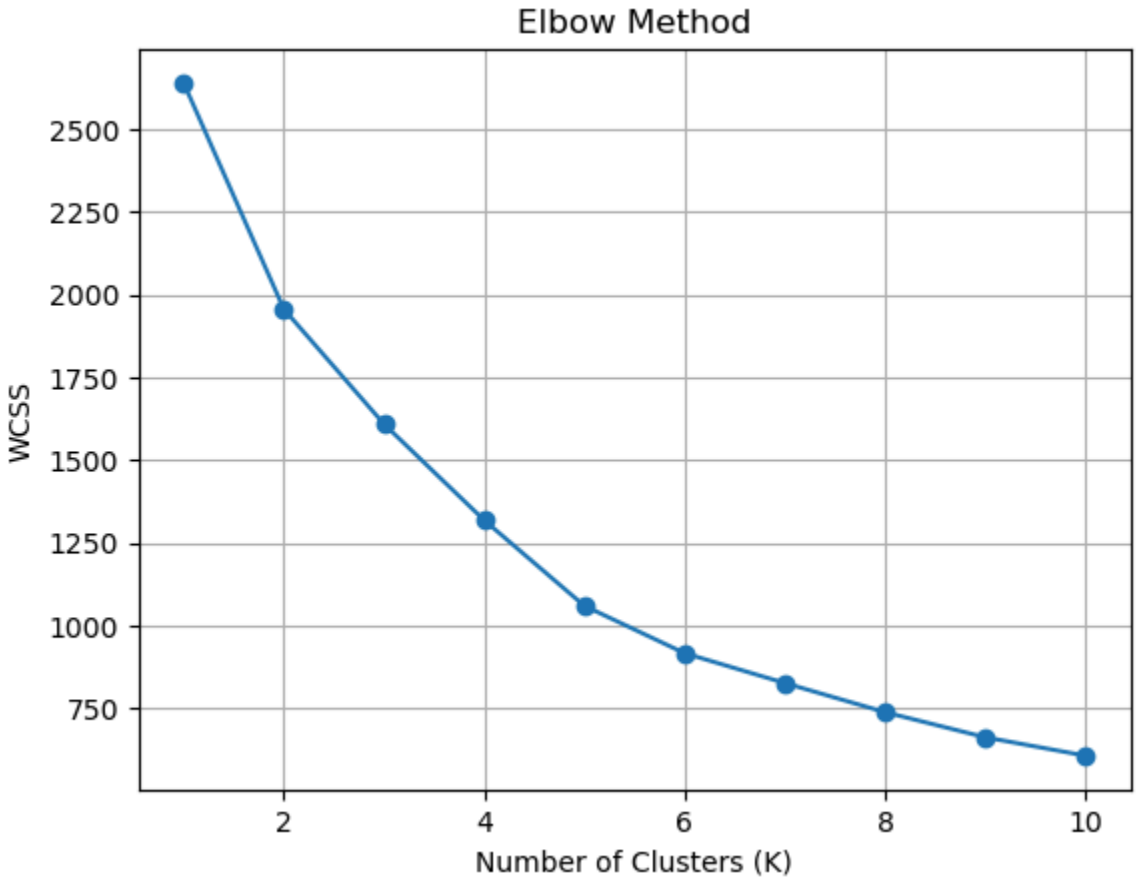
    wcss.append(kmeans.inertia_)

print("\n==== WCSS Values ====")
for k, value in enumerate(wcss, start=1):
    print(f"K = {k}: {value:.2f}")
```

```
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
```

```
==== WCSS Values ====
K = 1: 2640.00
K = 2: 1956.12
K = 3: 1608.43
K = 4: 1317.84
K = 5: 1058.77
K = 6: 915.74
K = 7: 825.40
K = 8: 737.39
K = 9: 661.45
K = 10: 605.92
```

In [9]:
```
plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.grid()
plt.show()
```



In [10…
```
scores = []

for k in range(2,11) :
    model = KMeans(
        n_clusters=k,
        n_init=10,
        random_state=42
```

```
    )

    labels = model.fit_predict(x_scale)

    score = silhouette_score(x_scale, labels)

    scores.append(score)

    print(f"Cluster {k} : Silhoutte Score = {score}")
```

```
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
Cluster 2 : Silhoutte Score = 0.5472146913441881
Cluster 3 : Silhoutte Score = 0.5482872649700601
Cluster 4 : Silhoutte Score = 0.34847138657014315
Cluster 5 : Silhoutte Score = 0.3690403826812808
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
Cluster 6 : Silhoutte Score = 0.3782288965882046
Cluster 7 : Silhoutte Score = 0.3342819788182625
Cluster 8 : Silhoutte Score = 0.3201161896109077
```

```
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
Cluster 9 : Silhoutte Score = 0.30902833452574485
Cluster 10 : Silhoutte Score = 0.3112143583568513
```

In [11…

```
best_k = range(2,11)[np.argmax(scores)]
print("Optimal Number Of Clusters :", best_k)
```

```
Optimal Number Of Clusters : 3
```

In [12…

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['Customer_Segmentation'] = kmeans.fit_predict(x_scale)

print("\n===== Customer Segmentation =====")
df[['Fresh', 'Milk', 'Grocery', 'Frozen',
    'Detergents_Paper', 'Delicassen', 'Customer_Segmentation']].head(10)
```

```
===== Customer Segmentation =====
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserWarnin
g: KMeans is known to have a memory leak on Windows with MKL, when there
are less chunks than available threads. You can avoid it by setting the e
nvironment variable OMP_NUM_THREADS=2.
  warnings.warn(
```

Out[12]:

	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicassen	Customer_Segmentation
0	12669	9656	7561	214	2674	1338	1
1	7057	9810	9568	1762	3293	1776	1
2	6353	8808	7684	2405	3516	7844	1
3	13265	1196	4221	6404	507	1788	1
4	22615	5410	7198	3915	1777	5185	1
5	9413	8259	5126	666	1795	1451	1
6	12126	3199	6975	480	3140	545	1
7	7579	4956	9426	1669	3321	2566	1
8	5963	3648	6192	425	1716	750	1
9	6006	11093	18881	1159	7425	2098	0

In [14…
```
print("\nNumber of Customers in Each Cluster:")
print(df['Customer_Segmentation'].value_counts().sort_index())
```

```
Number of Customers in Each Cluster:
Customer_Segmentation
0     45
1    393
2      2
Name: count, dtype: int64
```

In [16…
```
print("\nCluster Analysis:")
df.groupby('Customer_Segmentation')[
    ['Fresh', 'Milk', 'Grocery','Frozen',
      'Detergents_Paper', 'Delicassen']
    ].mean().round(2)
```

```
Cluster Analysis:
```

Out[16]:

Customer_Segmentation	Fresh	Milk	Grocery	Frozen	Detergents_Paper	Delicasse
0	10440.93	19386.42	28656.09	2190.24	13327.80	2374.2
1	12062.91	4115.10	5534.97	2940.68	1696.17	1299.1
2	34782.00	30367.00	16898.00	48701.50	755.50	26776.0

In [ ]:
```
```